Code -

```
#include <iostream>
#include <vector>
#include <chrono>
#include <omp.h>
using namespace std;

void merge(vector<int>& arr, int l, int m, int r) {
    int n1 = m - l + 1;
    int n2 = r - m;

    vector<int> L(n1), R(n2);

    for (int i = 0; i < n1; i++) L[i] = arr[l + i];
    for (int j = 0; j < n2; j++) R[j] = arr[m + 1 + j];

    int i = 0, j = 0, k = l;

    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) arr[k++] = L[i++];
        else arr[k++] = R[j++];
    }

    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
}

void mergeSortSequential(vector<int>& arr, int l, int r) {
    if (l < r) {
        int m = (l + r) / 2;
        mergeSortSequential(arr, l, m);
        mergeSortSequential(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}

void mergeSortParallel(vector<int>& arr, int l, int r) {
    if (r - l < 1000) {
        mergeSortSequential(arr, l, r);
        return;
    }

    int m = (l + r) / 2;

    #pragma omp task shared(arr)
    mergeSortParallel(arr, l, m);

    #pragma omp task shared(arr)
    mergeSortParallel(arr, m + 1, r);

    #pragma omp taskwait
    merge(arr, l, m, r);
}

int main() {
    int N;

    cout << "Enter the size of the vector: ";
    cin >> N;

    vector<int> arr(N), original(N);

    cout << "Enter the elements of the vector (space-separated): ";
```

```
    for (int i = 0; i < N; i++) {
        cin >> arr[i];
        original[i] = arr[i];
    }

    auto start = chrono::high_resolution_clock::now();
    mergeSortSequential(arr, 0, N - 1);
    auto end = chrono::high_resolution_clock::now();

    cout << "Sequential Merge Sort Time (ms): "
         << chrono::duration<double>(end - start).count() << endl;

    arr = original;

    start = chrono::high_resolution_clock::now();

    #pragma omp parallel
    {
        #pragma omp single
        mergeSortParallel(arr, 0, N - 1);
    }

    end = chrono::high_resolution_clock::now();

    cout << "Parallel Merge Sort Time (ms): "
         << chrono::duration<double>(end - start).count() << endl;

    return 0;
}
```

Output -

```
Enter the size of the vector: 5
Enter the elements of the vector (space-separated): 5 3 1 4 2
Sequential Merge Sort Time (ms): 9.84e-06
Parallel Merge Sort Time (ms): 2.03e-06


=== Code Execution Successful ===
```